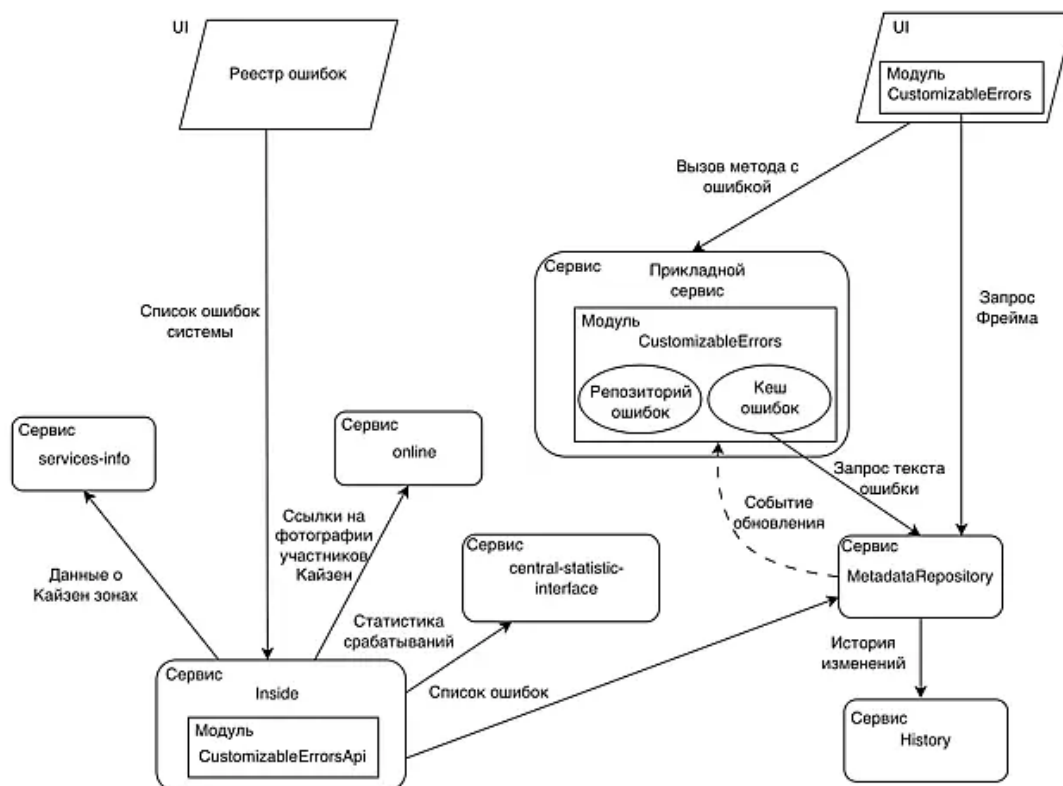


Архитектура

Схема взаимодействия сервисов



- Сервис MetadataRepository отвечает за хранение данных ошибки, верстки, текстов, а так же истории изменений
- Сервис Central-Statistic - отвечает за хранение статистики срабатывания ошибок
- Модуль CustomizableErrors(БЛ) - отвечает за выброс ошибки и передачу ее между сервисами.
- Модуль CustomizableErrors(UI) - отвечает за отображение ошибки пользователю.
- Модуль CustomizableErrorsAPI(БЛ) - отвечает за отдачу данных для реестр ошибок на inside.

Метатип "Кастомизируемая ошибка"

Главным объектом системы экземпляры [метатипа](#) "Кастомизируемая ошибка". В терминах C++, метатип ошибки можно представить в виде класса со следующим набором полей:

```
1  * @brief The ErrorInfo class - мета-данные ресурса ERRX
2  */
3  class CustomizableError
4  {
5      // Идентифкатор ошибки
6      Uuid mId;
7      // Идентифкатор кайзен зоны
8      Uuid mKaizenZone;
9      // Идентифкатор компонента, которому принадлежит ошибка
10     Uuid mComponentUuid;
11
12     // Имя типа прикладного объекта, определяющего контекст ошибки
13     boost::optional< String > mDataObject;
14 }
```

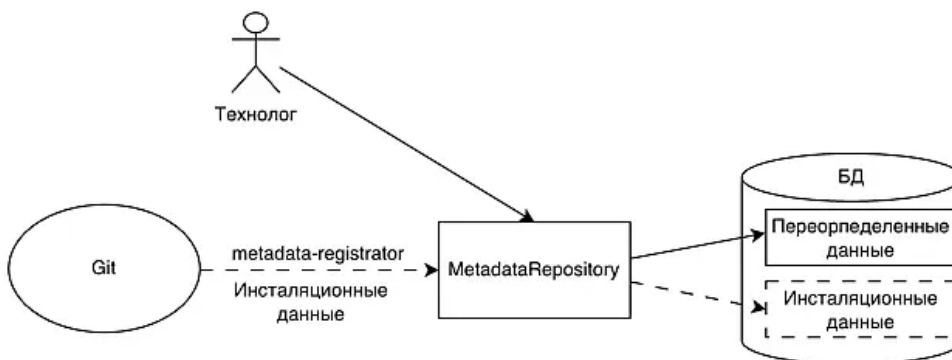
```

15 // Краткое название ошибки, заголовок
16 String mMessage;
17 // Рекомендации по исправлению ошибки
18 String mRecommendation;
19 // Комментарий, в котором описан сценарий возникновения ошибки
20 String mScenario;
21
22 // Фрейм ошибки
23 String mFrame;
24
25 // Тип ошибок. Enum: Error/Warning/Info
26 rs::ErrorType mErrorType;
27 };

```

Все прикладные ошибки, это экземпляры этого метатипа с заданными значениями полей.

Хранение метаданных ошибок

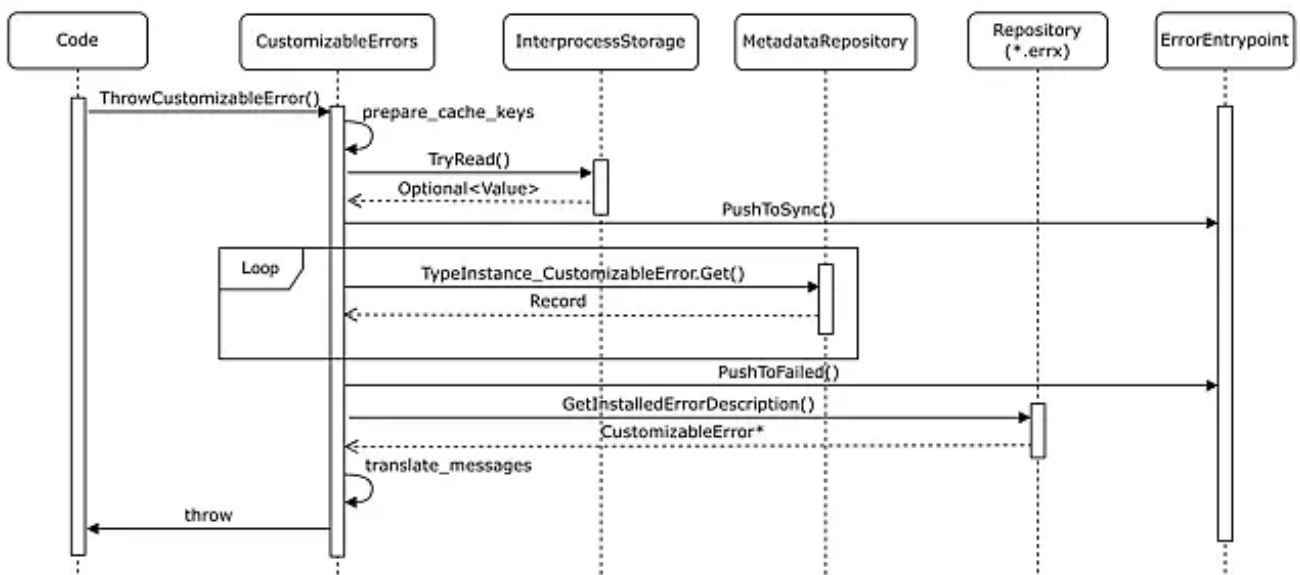


- Git - хранят в файлах *.errx инсталляционные данные ошибок, т.е. то представление ошибки, которое было верным в момент ее создания.
- MetadataRepository - хранит две версии ошибки, инсталляционную и переопределенную(может не быть), после загрузки ошибки в хранилище, единственным способом изменения ее представления является интерфейс "Админки ошибок" на inside. К представлению относятся поля: Тип, Сообщение, Рекомендация и Фрейм.

Синхронизация представления между MetadataRepository и Git односторонняя, Git->MetadataRepository, поэтому в Genie можно видеть не актуальную верстку.

Кеш метаданных ошибок

Диаграмма отображает наиболее полную последовательность вызовов при выбросе ошибки в коде



- Для кеширования данных ошибок используется межпроцессный кеш [InterprocessStorage](#).

- В случае отсутствия данных в кеше, данные об ошибке запрашиваются синхронно с таймаутом в 50мс с сервиса MetadataRepository. При ошибках связи запрос повторяется 1 раз.
- В случае отсутствия данных в кеше и ошибках при синхронном получении данных из MetadataRepository используются инсталляционные данные ошибки(из *.errx).
- Полученные текстовые данные ошибки локализируются, без генерации ошибки в лог в случае отсутствия перевода.

Для каждой ошибки в кеше хранится две записи:

1. Данные ошибки: ключ = строкифицированный идентификатор ошибки, значение `std::pair< std::map<String, String>, std::map<String, String> >`. Пара значений это "Сообщение" и "Рекомендация", в виде словаря с переводами `{ lang: translation }`.
2. Время инвалидации ошибки: ключ = `<строкифицированный идентификатор ошибки>_expired`, значение `DateTime`, время следующего обновления.

ErrorEntrypoint

Это [точка входа](#) для синхронизации данных кастомизируемых ошибок. Синхронизируются описания для трех групп ошибок:

1. Регулярная синхронизация - те ошибки, которые были брошены в процессе работы сервиса и при получении данных для которых не было проблем. Таймаут этой синхронизации - 3600с.
2. Синхронизации при ошибках - синхронизация тех ошибок, в процессе получения данных для которых возникли проблемы(например metadata-repository не успел отдать данные). Таймаут этой синхронизации - 60с.
3. Инвалидация данных - синхронизация ошибок, у которых поменялось описания в metadata-repository и было получено событие о необходимости обновления. Таймаут этой синхронизации - 60с.